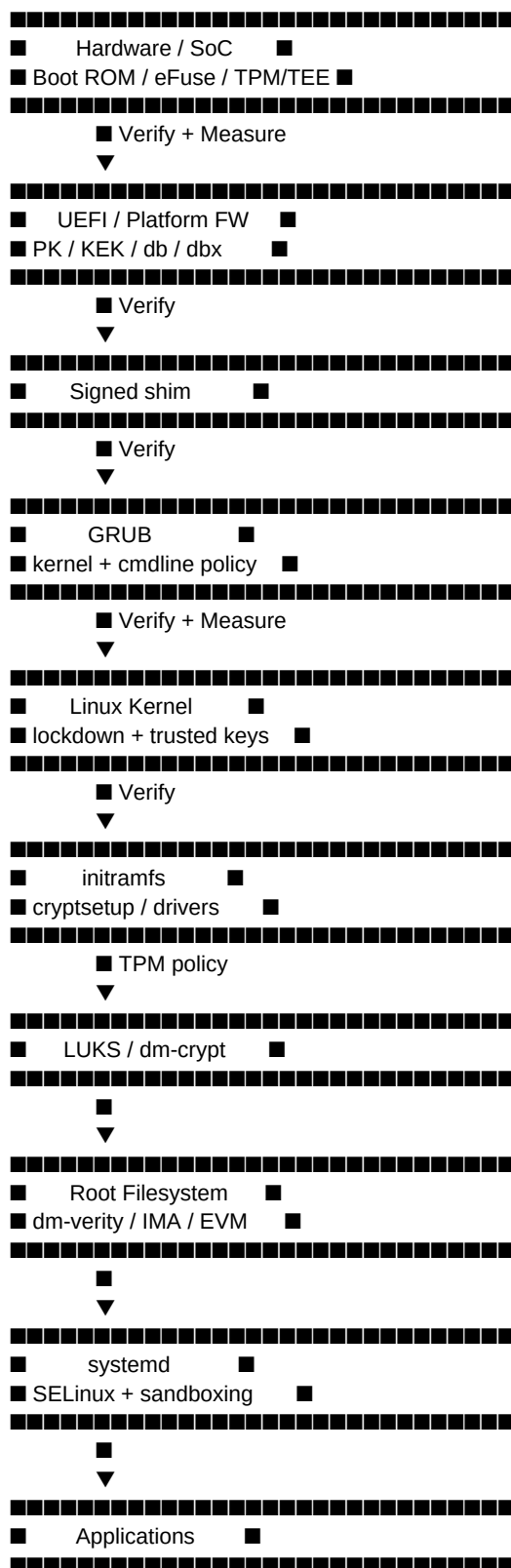


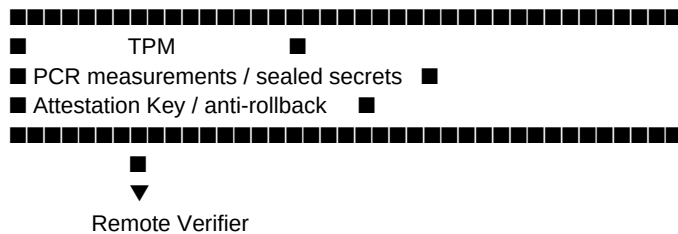
Secure Linux Boot Architecture — HLD, LLD & Code

Senior SDE / Kernel Engineer Interview Preparation

Scope: A production-oriented secure boot design covering hardware root of trust, UEFI Secure Boot, shim/GRUB, signed kernel and modules, measured boot with TPM, LUKS, dm-verity, IMA/EVM, lockdown, A/B updates, anti-rollback, recovery, and remote attestation.

1. HLD — High-Level Design





2. HLD Components and Responsibilities

Component	Responsibility
Boot ROM / Hardware RoT	Immutable first-stage trust anchor; verifies firmware.
UEFI Secure Boot	Authenticates EFI executables using PK/KEK/db/dbx.
Shim	Distribution-trusted bridge between UEFI and GRUB.
GRUB	Loads authenticated kernel/initramfs and protects boot configuration.
Linux Kernel	Kernel integrity, trusted keyrings, lockdown, module verification.
TPM	PCR measurement, sealed secrets, attestation, monotonic counters/policies.
LUKS/dm-crypt	Block-level confidentiality.
dm-verity	Integrity of immutable root filesystem.
IMA/EVM	Runtime measurement/appraisal of files.
systemd/SELinux	Least privilege, service isolation, mandatory access control.
A/B updater	Atomic updates, health checks, rollback to known-good image.

3. HLD Security Flow

1. Hardware establishes immutable root of trust.
2. Firmware verifies the next boot stage.
3. Every stage measures important artifacts into TPM PCRs.
4. UEFI Secure Boot verifies shim/EFI binaries.
5. Shim verifies GRUB according to distribution policy.
6. GRUB verifies/loads the signed kernel and authenticated initramfs.
7. Kernel enables lockdown and rejects unauthorized modules.
8. Initramfs evaluates TPM policy and unlocks LUKS only for an approved state.
9. dm-verity protects immutable rootfs; IMA/EVM can protect mutable files.
10. systemd starts isolated services under SELinux/AppArmor policy.
11. TPM quote can prove the measured state to a remote verifier.

4. LLD — Low-Level Design

4.1 Core Interfaces

```
class BootArtifact {
public:
    virtual ~BootArtifact() = default;
    virtual std::string name() const = 0;
    virtual std::vector<uint8_t> digest() const = 0;
    virtual std::vector<uint8_t> signature() const = 0;
};

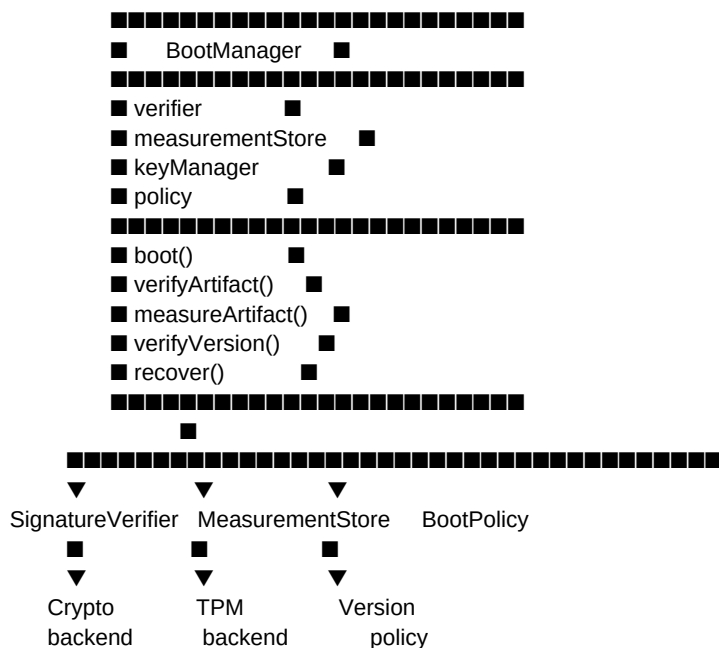
class SignatureVerifier {
public:
    virtual ~SignatureVerifier() = default;
    virtual bool verify(const BootArtifact& artifact) = 0;
};

class MeasurementStore {
public:
    virtual ~MeasurementStore() = default;
    virtual bool extendPCR(uint32_t pcr,
                          const std::vector<uint8_t>& digest) = 0;
};

class KeyManager {
public:
    virtual ~KeyManager() = default;
    virtual bool isTrusted(const std::vector<uint8_t>& keyId) = 0;
    virtual bool revoke(const std::vector<uint8_t>& keyId) = 0;
};

class BootPolicy {
public:
    virtual ~BootPolicy() = default;
    virtual bool allow(const BootArtifact& artifact) = 0;
    virtual bool antiRollback(uint64_t version) = 0;
};
```

4.2 Boot Manager Class Design



4.3 Detailed Boot Sequence

```

BootManager::boot()
|
+--> readActiveSlot()
|
+--> loadMetadata()
|
+--> verifyVersion()
|   |
|   +--> reject if version < minimumVersion
|
+--> verifyArtifact(shim/bootloader)
|
+--> measureArtifact(shim)
|
+--> verifyArtifact(kernel)
|
+--> verifyArtifact(initramfs)
|
+--> measureArtifact(kernel)
|
+--> measureArtifact(initramfs)
|
+--> verifyCommandLine()
|
+--> verifyTPMPolicy()
|
+--> unlockRootFS()
|
+--> verifyRootFS()
|
+--> markBootAttempt()
|
+--> handOffToKernel()

```

5. LLD — Data Structures

```

struct ArtifactMetadata {
    std::string name;
    uint64_t version;
    std::array<uint8_t, 32> sha256;
    std::vector<uint8_t> signature;
    std::string signingKeyId;
};

enum class BootSlot {
    A,
    B
};

struct SlotMetadata {
    BootSlot slot;
    uint64_t version;
    bool bootable;
    bool successful;
    uint32_t attempts;
};

struct TPMPolicy {
    uint32_t pcrIndex;
    std::array<uint8_t, 32> expectedDigest;
    uint64_t minimumVersion;
};

struct BootResult {
    bool success;
    std::string error;
};

```

6. C++ Reference Implementation

The following is interview-grade reference code. Real implementations delegate cryptography and TPM operations to established platform libraries rather than implementing cryptographic primitives themselves.

```
#include <array>
#include <cstdint>
#include <iostream>
#include <string>
#include <vector>

class SignatureVerifier {
public:
    bool verify(const std::string& image,
               const std::vector<uint8_t>& signature,
               const std::string& keyId)
    {
        // Real implementation:
        // - SHA-256/SHA-384 hash
        // - verify RSA/ECDSA/Ed25519 signature
        // - validate certificate/key chain
        // - check revocation
        std::cout << "Verifying " << image
                  << " using key " << keyId << "\n";

        return !image.empty() &&
               !signature.empty() &&
               !keyId.empty();
    }
};

class TPM {
public:
    bool extendPCR(uint32_t pcr,
                  const std::array<uint8_t, 32>& digest)
    {
        // Real implementation uses TPM2_PCR_Extend.
        std::cout << "Extending PCR " << pcr << "\n";
        return true;
    }

    bool authorizeUnlock(uint32_t pcr,
                        const std::array<uint8_t, 32>& expected)
    {
        // Real implementation uses TPM2 policy sessions
        // and sealed/unsealing operations.
        std::cout << "Checking TPM policy on PCR "
                  << pcr << "\n";
        return true;
    }
};

class RootFS {
public:
    bool unlock()
    {
        // Real implementation invokes cryptsetup/LUKS
        // or an equivalent storage subsystem.
        std::cout << "Unlocking encrypted root filesystem\n";
        return true;
    }

    bool verify()
    {
        // Real implementation configures dm-verity or
        // performs IMA/EVM appraisal as appropriate.
        std::cout << "Verifying root filesystem\n";
    }
};
```

```

        return true;
    }
};

class BootManager {
public:
    BootManager(SignatureVerifier& verifier,
                TPM& tpm,
                RootFS& rootfs)
        : verifier_(verifier),
          tpm_(tpm),
          rootfs_(rootfs)
    {}

    bool boot()
    {
        if (!verifyBootloader())
            return fail("bootloader verification failed");

        if (!verifyKernel())
            return fail("kernel verification failed");

        if (!verifyInitramfs())
            return fail("initramfs verification failed");

        if (!measureBoot())
            return fail("TPM measurement failed");

        if (!authorizeDiskUnlock())
            return fail("TPM authorization failed");

        if (!rootfs_.unlock())
            return fail("rootfs unlock failed");

        if (!rootfs_.verify())
            return fail("rootfs integrity verification failed");

        std::cout << "Secure boot successful\n";
        return true;
    }

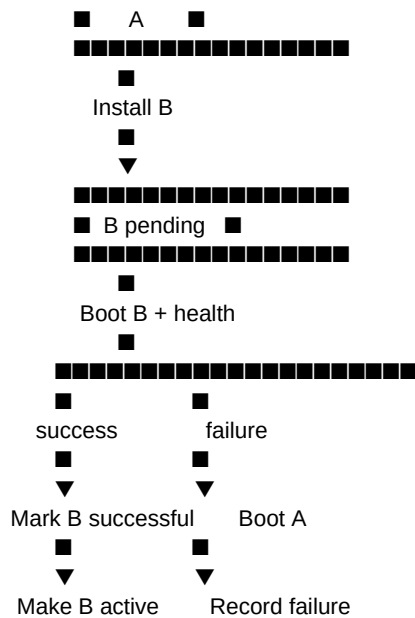
private:
    bool verifyBootloader()
    {
        return verifier_.verify(
            "grub.efi",
            {0x01, 0x02, 0x03},
            "BOOTLOADER_SIGNING_KEY");
    }

    bool verifyKernel()
    {
        return verifier_.verify(
            "vmlinuz",
            {0x04, 0x05, 0x06},
            "KERNEL_SIGNING_KEY");
    }

    bool verifyInitramfs()
    {
        return verifier_.verify(
            "initramfs.img",
            {0x07, 0x08, 0x09},
            "INITRAMFS_SIGNING_KEY");
    }

    bool measureBoot()

```

10. Secure Update Pseudocode

```

bool installUpdate(const Image& image)
{
    if (!verifySignature(image))
        return false;

    if (!checkVersion(image.version))
        return false;

    if (!verifyCompatibility(image))
        return false;

    Slot inactive = getInactiveSlot();

    if (!writeImage(inactive, image))
        return false;

    if (!verifyWrittenImage(inactive))
        return false;

    markPending(inactive);

    reboot();

    return true;
}

void onSuccessFullBoot()
{
    Slot slot = currentSlot();

    if (healthCheckPassed()) {
        markSuccessful(slot);
        makeActive(slot);
        resetBootAttempts(slot);
    }
}

```


11. Linux Implementation Mapping

Security requirement	Linux/platform mechanism
UEFI binary authenticity	UEFI Secure Boot, PK/KEK/db/dbx
Linux bootloader trust	shim + signed GRUB
Kernel authenticity	Signed kernel + trusted keyrings
Module authenticity	CONFIG_MODULE_SIG + enforcement
Kernel post-boot protection	Kernel Lockdown
Disk confidentiality	LUKS / dm-crypt
Measured boot	TPM 2.0 PCRs
Key release	TPM policy / sealed objects
Immutable rootfs integrity	dm-verity
Mutable file integrity	IMA/EVM
Mandatory access control	SELinux/AppArmor
Service isolation	systemd sandboxing/capabilities
Rollback protection	TPM NV/hardware monotonic counter + metadata
Atomic upgrade	A/B slots
Remote proof	TPM quote / remote attestation

12. Interview Trade-offs

Secure Boot only

- + Strong prevention before kernel execution
- Does not prove runtime state to a remote verifier

Measured Boot

- + Provides evidence of boot state
- Does not itself prevent execution of measured-but-untrusted code

dm-verity

- + Excellent immutable filesystem integrity
- Poor fit for frequently changing filesystems

IMA/EVM

- + Supports mutable files and runtime appraisal
- More policy/configuration complexity

LUKS

- + Protects confidentiality at rest
- Does not provide software integrity by itself

A/B updates

- + Robust recovery from failed upgrades
- Requires additional storage and careful state management

13. Senior-Level Interview Questions

- What is the difference between Secure Boot and Measured Boot?
- Why is TPM needed if UEFI Secure Boot already exists?
- How does PCR extend work?
- How would you prevent an attacker from booting an older signed kernel?
- How would you protect the initramfs?

- How does kernel lockdown complement Secure Boot?
- How do signed kernel modules work?
- How would you design a secure A/B update system?
- What happens if the kernel signing key is compromised?
- How would a remote server verify that a Linux machine booted an approved image?
- Why use dm-verity instead of a normal filesystem hash?
- How would you securely recover from a failed kernel update?

14. Key Interview Takeaway

Root of Trust → Verify → Measure → Protect Keys → Encrypt Storage → Verify Root FS → Lock Down Kernel → Isolate Services → Attest → Securely Update.

For an Amazon Senior SDE / Kernel interview, the strongest answer is not just the boot sequence. Explain the trust boundaries, attacker model, cryptographic verification, TPM measurements, failure handling, key rotation, rollback protection, and recovery strategy.